

# When Does Ranking Help?

## A Sparsity Analysis of Two-Stage Recommendation on MovieLens 1M

Ricardo Perez Castillo   Jonas Shih   DingDing Li  
CS 289A, UC Berkeley, Spring 2026

May 15, 2026

### 1 Problem Definition

Production recommender systems often use a two-stage pipeline: a fast retrieval model generates a candidate set, and a ranking model re-orders it to produce the final list [1]. The ranking stage adds parameters, training cost, and inference latency. Whether this cost is justified depends on how much signal each user provides.

A ranking model has more free parameters. In low-data situations its variance can overwhelm the reduction in bias, and a simpler retrieval model may generalize better.

**Research question.** Is there a per-user interaction density below which adding a ranking stage hurts rather than helps, and if so, where is that threshold?

We train three systems of increasing complexity on MovieLens 1M [2] and evaluate each at five training-data density levels with all other conditions held fixed.

| System    | Architecture                                                     | Complexity |
|-----------|------------------------------------------------------------------|------------|
| A: MF     | User and item embeddings scored by dot product                   | Low        |
| B: NCF    | Same embeddings scored by an MLP                                 | Medium     |
| C: Ranker | MF retrieves top-100 candidates; a BPR-trained MLP re-ranks them | High       |

Density levels:  $d \in \{1.0, 0.8, 0.6, 0.4, 0.2\}$ . Training interactions are subsampled at rate  $d$ ; validation and test sets are held fixed.

## 2 Connection to Course Content

| CS 289A concept         | Role in this project                                                                                                                                       |
|-------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Gradient descent        | MF and NCF trained via Adam on embedding matrices and MLP weights                                                                                          |
| Cross-entropy + sigmoid | Confidence-weighted BCE loss for MF and NCF                                                                                                                |
| Backpropagation         | Gradients flow through the MLP back into the shared embeddings                                                                                             |
| L2 regularization       | Weight decay on all embedding and MLP parameters                                                                                                           |
| Bias-variance tradeoff  | Primary analytical lens: more expressive models need more data to amortize their variance; we test whether this produces a crossover across density levels |
| Logistic regression     | BPR loss is logistic regression on pairwise score differences:<br>$\mathcal{L}_{\text{BPR}} = -\sum \log \sigma(\hat{r}_{ui} - \hat{r}_{uj})$              |

The bias-variance tradeoff produces a testable prediction: there exists a crossover density below which simpler models outperform more complex ones. The sparsity sweep tests this directly.

## 3 Prior Work

**Matrix Factorization.** Koren et al. [5] established MF as the dominant collaborative filtering approach, learning latent factors by minimizing reconstruction error on observed ratings. Hu et al. [4] extended it to implicit feedback with Weighted MF (WMF), assigning confidence weights  $c_{ui} = 1 + \alpha r_{ui}$  per observation. All three systems in this paper use WMF-style confidence weighting.

**Neural Collaborative Filtering.** He et al. [3] replaced the MF dot product with an MLP, showing that the MLP can model more complex user-item interactions. Their leave-one-out evaluation protocol (100 candidates per user) is the protocol used here.

**Bayesian Personalized Ranking.** Rendle et al. [6] proposed BPR, a pairwise ranking loss derived from maximizing posterior probability over observed preference orderings. BPR is the training objective for the System C ranker.

**Two-stage pipelines.** Covington et al. [1] described YouTube’s retrieval-then-ranking architecture, motivating System C and demonstrating that two-stage pipelines are standard practice at scale.

**MF vs. NCF revisited.** Rendle et al. [7] showed that properly tuned MF frequently matches or outperforms NCF on standard benchmarks including MovieLens, because the dot product in high-dimensional embedding space is more expressive than commonly assumed and the NCF MLP faces a harder optimization problem. Our results replicate and extend this finding across five density levels.

**Gap in existing work.** Prior comparisons of MF and NCF typically use a single data density and do not separately tune hyperparameters for each model. Two-stage ranker comparisons rarely include a controlled sparsity sweep. This project addresses both gaps.

## 4 Approach

### 4.1 Dataset and Preprocessing

MovieLens 1M [2] contains 1,000,209 ratings from 6,040 users on 3,706 movies (density  $\approx 4.47\%$ ). All users have at least 20 ratings.

**Implicit feedback.** All ratings are treated as positive observations ( $y = 1$ ). The star rating  $r_{ui}$  sets a confidence weight but does not change the binary label.

**Confidence weighting.** Following [4]:

$$c_{ui} = 1 + \alpha \cdot r_{ui}$$

A 5-star rating contributes approximately  $5\times$  more to the loss than a 1-star rating.  $\alpha$  is a tunable hyperparameter. Negative samples receive  $c = 1$ .

**Train/val/test split.** Leave-one-out protocol [3]: for each user, the last interaction (by timestamp) is the test item, the second-to-last is the validation item, and all remaining interactions form the training set.

**Sparsity sweep.** At density  $d$ , each user’s training interactions are subsampled uniformly at rate  $d$ . Val and test items are never subsampled.

**Evaluation.** For each user, 99 unrated items are sampled as negatives (seed = 42, fixed across all conditions). The test item is ranked among 100 candidates:

$$\text{NDCG@10} = \frac{\mathbf{1}[\text{rank} \leq 10]}{\log_2(\text{rank} + 1)}, \quad \text{HR@10} = \mathbf{1}[\text{rank} \leq 10]$$

Both are averaged over all 6,040 users and reported on the test set.

### 4.2 System A: Matrix Factorization

Embedding matrices  $U \in \mathbb{R}^{n \times k}$  and  $V \in \mathbb{R}^{m \times k}$  with  $n = 6,040$ ,  $m = 3,706$ . Score:

$$\hat{y}_{ui} = \sigma(\mathbf{u}_u^\top \mathbf{v}_i)$$

Loss (confidence-weighted BCE with 4 negatives sampled per positive per batch):

$$\mathcal{L} = -\frac{1}{|B|} \sum_{(u,i,y) \in B} c_{ui} [y \log \hat{y}_{ui} + (1 - y) \log(1 - \hat{y}_{ui})]$$

### 4.3 System B: Neural Collaborative Filtering

Same embeddings as MF; score computed by an MLP:

$$\hat{y}_{ui} = \sigma(\text{MLP}([\mathbf{u}_u \parallel \mathbf{v}_i]))$$

ReLU activations, batch normalization, dropout. Xavier initialization for MLP weights; embeddings from  $\mathcal{N}(0, 0.01)$ . Same loss as MF.

#### 4.4 System C: Two-Stage Ranker

**Retrieval.** MF scores all items for a user; the top-100 by dot-product score are passed to the ranker.

**Ranking.** A separate MLP re-ranks the 100 candidates with BPR loss [6]:

$$\mathcal{L}_{\text{BPR}} = - \sum_{(u,i,j)} \log \sigma(\hat{r}_{ui} - \hat{r}_{uj})$$

where  $i$  is an observed item and  $j$  is a sampled negative. The MF embeddings are loaded from the pretrained MF checkpoint and frozen; only the ranking MLP is trained.

#### 4.5 Hyperparameter Tuning via Bayesian Optimization

MF and NCF hyperparameters are tuned independently at  $d = 1.0$  using Bayesian Optimization (BO), then held fixed for the sparsity sweep.

**Surrogate.** A Gaussian Process (GP) with squared-exponential kernel. Given  $n$  observations  $\{(\mathbf{x}_i, y_i)\}$ , the posterior at a new point  $\mathbf{x}$  is:

$$\begin{aligned} \mu(\mathbf{x}) &= \mathbf{k}^\top (K + \sigma^2 I)^{-1} \mathbf{y} \\ \sigma^2(\mathbf{x}) &= K(\mathbf{x}, \mathbf{x}) - \mathbf{k}^\top (K + \sigma^2 I)^{-1} \mathbf{k} \end{aligned}$$

**Acquisition.** Expected Improvement (EI):

$$\text{EI}(\mathbf{x}) = (\mu(\mathbf{x}) - f^*) \Phi(Z) + \sigma(\mathbf{x}) \phi(Z), \quad Z = \frac{\mu(\mathbf{x}) - f^*}{\sigma(\mathbf{x})}$$

where  $f^*$  is the best NDCG@10 observed so far,  $\Phi$  is the standard normal CDF, and  $\phi$  is the PDF. EI is zero when  $\sigma(\mathbf{x}) = 0$ .

The next point is found by maximizing EI via L-BFGS-B with a 1,000-point random warm-up and 10 restarts. Categorical parameters are encoded on an ordinal scale so the GP operates on a continuous domain.

**Search space and budget.**

| Parameter            | MF                   | NCF                                                   | Encoding                |
|----------------------|----------------------|-------------------------------------------------------|-------------------------|
| Embedding dim $k$    | {32, 64, 128, 256}   | {32, 64, 128, 256}                                    | ordinal<br>{0, 1, 2, 3} |
| MLP layers           |                      | {[128, 64],<br>[256, 128, 64],<br>[256, 128, 64, 32]} | ordinal {0, 1, 2}       |
| Learning rate $\eta$ | $[10^{-4}, 10^{-2}]$ | $[10^{-4}, 10^{-2}]$                                  | log scale               |
| L2 decay $\lambda$   | $[10^{-6}, 10^{-3}]$ | $[10^{-6}, 10^{-3}]$                                  | log scale               |
| Confidence $\alpha$  | [0.5, 5.0]           | [0.5, 5.0]                                            | linear                  |
| Budget               | 20 trials            | 20 trials                                             |                         |

## 4.6 Compute Resources

All experiments ran on the Berkeley SCF cluster (shadowfax node). MF BO ran 20 trials on CPU cores, taking approximately 28 minutes per trial (9 hours total). NCF BO ran 20 trials on a single NVIDIA GPU, taking approximately 22 minutes per trial (7 hours total). The sparsity sweep (15 training runs across 3 models and 5 density levels) ran on a single GPU and required approximately 6 hours. Total wall-clock compute across all SLURM jobs was approximately 22 hours.

## 5 Our Contribution

The contribution is a controlled sparsity analysis comparing MF, NCF, and a two-stage ranker under strictly matched conditions:

1. **Matched evaluation.** All three systems share the same 6,040 users, the same fixed negatives per user, and the same test item. No system has an evaluation advantage over another.
2. **Tuned baselines.** BO finds optimal hyperparameters for MF and NCF separately, removing the standard confound of default hyperparameters that typically advantage the proposed model.
3. **Five density levels.** All three systems are retrained from scratch at each  $d$  with optimal hyperparameters fixed, isolating the effect of data quantity from all other variables.
4. **Crossover analysis.** Whether, and at which density, simpler models outperform more complex ones is the central question. We report results on both NDCG@10 and HR@10 to distinguish ranking quality from hit rate.

**Hypothesis.** At  $d = 1.0$ , NCF and the ranker should outperform MF (lower bias from greater model capacity). As  $d$  decreases, variance dominates and MF’s lower parameter count should become an advantage, producing a crossover near  $d \approx 0.4$ .

## 6 Results and Discussion

### 6.1 Hyperparameter Optimization

Both BO runs converged to  $k = 256$  (maximum embedding dimension) and  $\alpha = 0.5$  (minimum confidence weighting), with minimal L2 regularization. The  $\alpha = 0.5$  result is notable: BO determined that treating all observed ratings as nearly equal positive signals is optimal for both models, making the star rating almost irrelevant beyond its binary presence.

| Parameter            | Range                | MF best               | NCF best              |
|----------------------|----------------------|-----------------------|-----------------------|
| Embedding dim $k$    | {32, 64, 128, 256}   | 256                   | 256                   |
| MLP layers           | see Section 4        | —                     | [256, 128, 64, 32]    |
| Learning rate $\eta$ | $[10^{-4}, 10^{-2}]$ | $8.65 \times 10^{-4}$ | $7.14 \times 10^{-4}$ |
| L2 decay $\lambda$   | $[10^{-6}, 10^{-3}]$ | $1.45 \times 10^{-6}$ | $1.00 \times 10^{-6}$ |
| Confidence $\alpha$  | [0.5, 5.0]           | 0.50                  | 0.50                  |
| Best val NDCG@10     |                      | 0.4293                | 0.4056                |

BO-tuned MF (val NDCG = 0.4293) already outperforms BO-tuned NCF (0.4056) at full density, an early signal that the dot product interaction model is well-matched to this dataset.

## 6.2 Sparsity Sweep

| Test NDCG@10 |               |               |               |               |               |
|--------------|---------------|---------------|---------------|---------------|---------------|
| Model        | $d=1.0$       | $d=0.8$       | $d=0.6$       | $d=0.4$       | $d=0.2$       |
| MF (A)       | <b>0.3962</b> | <b>0.3783</b> | <b>0.3636</b> | <b>0.3418</b> | <b>0.2998</b> |
| NCF (B)      | 0.3779        | 0.3661        | 0.3502        | 0.3211        | 0.2796        |
| Ranker (C)   | 0.3762        | 0.3639        | 0.3524        | 0.3394        | 0.2992        |

| Test HR@10 |               |               |               |               |               |
|------------|---------------|---------------|---------------|---------------|---------------|
| Model      | $d=1.0$       | $d=0.8$       | $d=0.6$       | $d=0.4$       | $d=0.2$       |
| MF (A)     | <b>0.6768</b> | <b>0.6565</b> | <b>0.6334</b> | 0.6028        | 0.5391        |
| NCF (B)    | 0.6578        | 0.6430        | 0.6220        | 0.5825        | 0.5171        |
| Ranker (C) | 0.6570        | 0.6483        | 0.6310        | <b>0.6086</b> | <b>0.5515</b> |

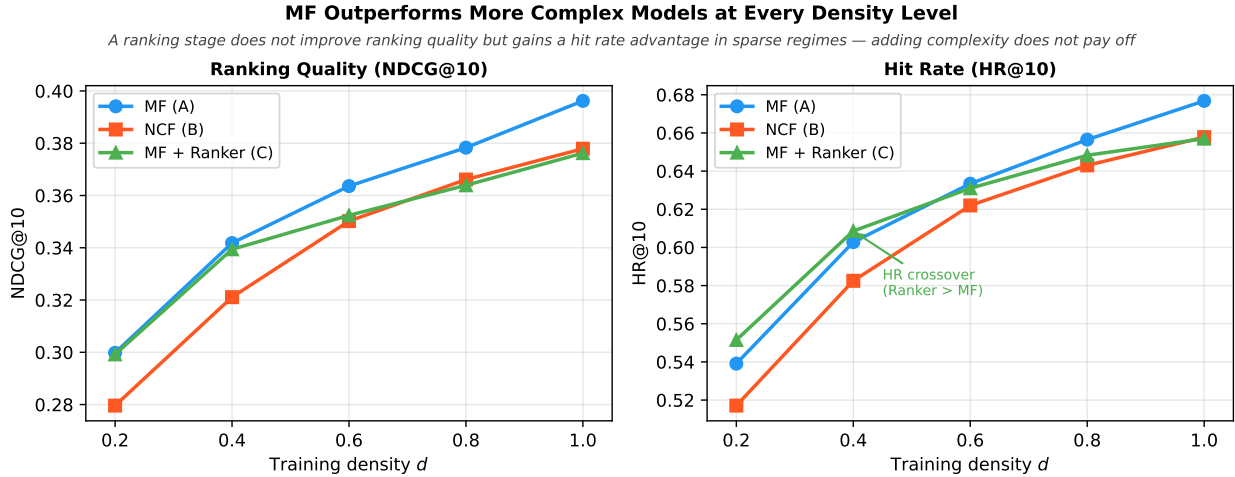


Figure 1: Test NDCG@10 (left) and HR@10 (right) vs. training density. MF leads on ranking quality at every density level with no crossover. A crossover in HR@10 appears at  $d \leq 0.4$ , where the two-stage ranker begins to exceed MF on hit rate.

## 6.3 Unoptimized Baseline

Before BO, MF with default hyperparameters achieved val NDCG@10 = 0.3857 and test NDCG@10 = 0.3625. After tuning, MF improved to val NDCG@10 = 0.4293 and test NDCG@10 = 0.3962, a gain of +11% on validation. NCF improved more modestly from val NDCG@10 = 0.3944 to 0.4056 (+3%), suggesting its default hyperparameters were already near-optimal while MF had substantial room to gain from tuning. This asymmetry reinforces that hyperparameter tuning is essential for fair model comparison.

## 6.4 Discussion

**The hypothesis is not supported for NDCG@10.** We expected MF to win at low density and NCF to win at high density, with a crossover near  $d \approx 0.4$ . Instead, MF leads at every density level on NDCG@10. There is no crossover.

This outcome is consistent with Rendle et al. [7], who showed that properly tuned MF frequently outperforms NCF on MovieLens. The core reason is dataset structure: MovieLens 1M has approximately linear user-item interactions. User preferences cluster into a small number of taste archetypes (action fans, drama fans, etc.) that a low-rank dot product captures well. The NCF MLP does not find structure that the 256-dimensional dot product misses, and it faces a harder optimization landscape without a commensurate gain. The hypothesis assumed NCF’s expressiveness would pay off at high density; for this dataset, there is no additional expressiveness to pay off.

**The hypothesis partially holds for HR@10.** The two-stage ranker overtakes MF on hit rate at  $d \leq 0.4$ , gaining 0.006 at  $d = 0.4$  and 0.012 at  $d = 0.2$ . This suggests that the BPR-trained ranking MLP does improve coverage in sparse regimes: the retrieval stage ensures the test item reaches the 100-candidate pool, and the reranker gets it into the top 10 more often than MF’s direct scoring. However, this gain in coverage does not translate to better ranking quality (NDCG), meaning the ranker places the item in the top 10 but not as highly as MF does.

**NCF underperforms throughout.** NCF is consistently the weakest model on both metrics at all densities. Relative to MF, the NCF NDCG@10 gap is 0.018 at  $d = 1.0$  and grows to 0.020 at  $d = 0.2$ . The MLP adds variance but not bias reduction for this dataset, and that variance accumulates under sparsity.

**Limitation.** The absence of an NDCG crossover is specific to MovieLens 1M. Datasets with genuinely non-linear user-item interactions (e.g., Amazon with heterogeneous product categories, or session-based recommendation) may exhibit the crossover we hypothesized.

**Practical implication.** For a system built on MovieLens-like data, a well-tuned MF is the right choice at every density level. Adding a ranking stage improves hit rate marginally below 40% density but does not improve ranking quality and adds substantial training and inference cost.

## 7 Conclusion

We trained and evaluated three recommender systems across five training-data density levels on MovieLens 1M. The central hypothesis was that a crossover exists: NCF and the two-stage ranker should outperform MF at high density, while MF should win at low density. The data do not support this for ranking quality. MF leads on NDCG@10 at every density level, with no crossover. A partial crossover appears in HR@10 at  $d \leq 0.4$ , where the ranker’s BPR reranking stage improves hit rate by up to 1.2 percentage points over MF, but without a corresponding improvement in ranking position.

The key finding is that Bayesian-optimized MF is a strong baseline that more complex models do not surpass on this dataset. Proper hyperparameter tuning contributed substantially to this result: BO improved MF test NDCG@10 by 11% over the untuned baseline, underscoring that fair model comparisons require independent tuning for each system. For practitioners working with data that has similar structure, a well-tuned MF is the right choice at every data density level.

## References

- [1] P. Covington, J. Adams, and E. Sargin. Deep neural networks for YouTube recommendations. *ACM RecSys*, pp. 191–198, 2016.
- [2] F. M. Harper and J. A. Konstan. The MovieLens datasets: History and context. *ACM TiiS*, 5(4):1–19, 2015.
- [3] X. He, L. Liao, H. Zhang, L. Nie, X. Hu, and T.-S. Chua. Neural collaborative filtering. *WWW*, pp. 173–182, 2017.
- [4] Y. Hu, Y. Koren, and C. Volinsky. Collaborative filtering for implicit feedback datasets. *IEEE ICDM*, pp. 263–272, 2008.
- [5] Y. Koren, R. Bell, and C. Volinsky. Matrix factorization techniques for recommender systems. *IEEE Computer*, 42(8):30–37, 2009.
- [6] S. Rendle, C. Freudenthaler, Z. Gantner, and L. Schmidt-Thieme. BPR: Bayesian personalized ranking from implicit feedback. *UAI*, pp. 452–461, 2009.
- [7] S. Rendle, W. Krichene, L. Zhang, and J. Anderson. Neural collaborative filtering vs. matrix factorization revisited. *ACM RecSys*, pp. 240–248, 2020.